Universidad del País Vasco Euskal Herriko Unibertsitatea
INFORMATIKA FAKULTATEA
FACULTAD DE INFORMÁTICA

Proiektua: Parte I Front-End

Laburpena

- Talde-lana, web sistemetako oinarrizko teknologiak erabiliz: `HTML`, `CSS` eta `JavaScript`.
- API publiko batetik abiatuta aplikazio bat eraikitzea.
- Zati honen gehieneko puntuazioa: 10 puntu (ikus ebaluazio-taula, **3. Taula**).
- Bi entrega (`milestone`): bakoitzak bere memoria eta `Git`-eko `tag`-a izango ditu.

Testuingurua eta helburuak

Lehen zati honetan, `JavaScript` bidezko web-aplikazio bat eraikiko duzue. Aplikazioak API publiko bat kontsumituko du, eta elementu-bilduma bat nabigatzailean gordetzeko aukera emango dizue.

Aplikazioa esploratzaile bat eta bilduma pertsonal bat izango da: erabiltzaileak APIko elementuak bilatuko ditu, bere bilduman gordeko ditu balorazio eta iruzkin propio batekin, eta, ondoren, elementuak iragazi, ordenatu, kudeatu eta aztertu ahal izango ditu.

Praktika honetan landu beharreko alderdiak:

- `JavaScript`-en oinarriak: objektuetara bideratutako programazioa (klaseak) eta programazio funtzionala (`filter`, `map`, `reduce`, `sort`, etabar).
- Nabigatzailea: `DOM`-aren manipulazioa, gertaeren kudeaketa eta tenporizadoreak.
- Komunikazio asinkronoa: `JSON`, promesak, `fetch` eta `async/await`.
- Bezeroaren aldeko iraunkortasuna: `localStorage`.

OHARRA: `Milestone 1` entregan aurkeztutako kodea Back-End zatiaren abiapuntua izango da.

`Milestone 1`eko emaitzaren adibidea: https://starwars.garmendia.eus/

`Milestone 2`ko emaitzaren adibidea: https://starwarsmoderno.garmendia.eus/

0. urratsa: APIa hautatzea

Aukeratu elementue zerrenda bat xehetasunekin itzultzen duen API publiko bat. Programatzen hasi aurretik, egiaztatu funtzionatzen duela eta haren datuak ulertzen dituzuela.

Elementua (`item`) APIak itzultzen dituen banakako elementuetako bakoitza da. Adibidez, liburuen API batean `item` bat liburu bat da; pertsonaien API batean, `item`-ak pertsonaia desberdinak dira.

APIaren baldintzak:

- Elementuak zerrendatzeko edo bilatzeko `endpoint` bat izatea.
- Erantzunak `JSON` formatuan ematea (ez XML, ez scraping).
- Autentifikazio konplexurik ez izatea: API `key` sinple bat onargarria da, baina `OAuth` ez.
- Elementu bakoitzak irudi adierazgarri baten `URL`a izatea.
- Zerrendan agertzen ez den API bat proposatuz gero, hasi aurretik irakaslearekin baliozkotu behar duzue.

APIa	Gaia	Key-a behar du?
TMDb	Filmak eta telesailak	Bai
RAWG	Bideo-jokoak	Bai
TheMealDB	Errezetak	Ez
PokéAPI	Pokemonak	Ez
Rick and Morty API	Pertsonaiak	Ez
OpenWeatherMap	Eguraldia eta hiriak	Bai
NASA API	Irudiak eta espazio-datuak	Bai

1. Taula: Baliogarriak diren API publiko batzuk

1. Entrega

Taldeak eGelan dagoen APIaren fitxa bete eta **2026-10-23** egunerako entregatu behar du.

Abiapuntua

Erabiliko duzuen APIa aukeratu ondoren, web-orri bat egingo duzue API horretako elementuak bilatu eta biltzeko. Ez zarete hutsetik hasiko; proiektu hau erabiliko duzue:

https://github.com/nbarrena/bildumakWS

Proiektuaren egitura:

```
proiektua/
├── index.html
├── css/
│   └── styles.css
├── js/
│   ├── app.js
│   ├── item.js
│   ├── bildumaStorage.js
│   └── api.js
└── templates/
    ├── bilatzaile.html
    ├── bilduma.html
    ├── hasiera.html
    └── kudeatuBilduma.html
```

Nola abiarazi: `VS Code`-ko `Live Server` erabiliz edo `npx serve` komandoa exekutatuz proiektuaren karpetatik.

Aurrez konponduta datorrena:

- `index.html`: `<nav>` elementua eta `Hasiera`, `Bilatzailea`, `NireBilduma` eta `Bilduma Kudeatu` atalen edukiontziak.
- `js/app.js`: txantiloien karga, atalak erakutsi/ezkutatzea eta menuko `active` klasea.
- `templates/`: aurreko lau atalen `HTML` egitura.
- `js/item.js`: `Item` klasearen eraikitzailea, oinarri gisa.
- `css/styles.css`: gutxieneko estiloak.

1. urratsa: Elementuak bilatzea (`bilatzaile.html`)

Martxan jarriko duzuen lehen atala bilatzailea da (`templates/bilatzaile.html`). Hiru eremu ditu dagoeneko:

- `Bilaketa`: zer bilatu nahi den adierazteko formularioa.
- `Emaitzak`: APIak itzultzen duen elementu-zerrenda.
- `Hautatutako elementua`: emaitzen eskuinaldean dagoen `div` panela; bertan, emaitzetan hautatutako elementuaren propietate guztiak ikusiko dira, eta bilduman gordetzeko aukera egongo da.

Atal honetan APIko bilaketa, emaitza bakoitza irudikatzen duen `Item` klasea eta elementuak bilduman gordetzea landuko dituzue.

1.1. APIan bilatzea (`js/api.js` eta `templates/bilatzaile.html`)

Inplementatu `bilatuApi()` funtzioa eta bilatzailearen logika:

- Bilaketa formularioa bidaltzean abiaraziko da (botoia sakatuta edo `Enter` tekla erabilita), ez testua idazten ari zareten bitartean.
- Bilaketa mota desberdinak eskain ditzakezue (izenburuaren arabera, `ID`aren arabera, etabar.), eta zuen elementu motarako zentzuzkoak direnak. Adibidez, testu-eremuaren ondoan hautatzaile bat jar dezakezue. Erabili APIaren fitxan dokumentatu dituzuen bilaketa motak.
- Erabili `fetch`.
- Erakutsi karga-egoera bat eta mezu argi bat emaitzarik ez dagoenean edo APIak huts egiten duenean.
- Erakutsi emaitzak modu sinplean (adibidez, elementu bakoitzaren izena), bilaketak funtzionatzen duela egiaztatzeko.

1.2. Item klasea (`js/item.js`)

Emaitza bakoitza behar bezala erakusteko, modu egituratuan irudikatu behar duzue. Osatu `Item` klasea: APIaren `JSON`a jasoko du, fitxan adierazitako eremuak gordeko ditu eta erabiltzailearen eremu propioak gehituko ditu (`balorazioa`, `iruzkina`, `gordetakoData`).

- `toHTML()`: elementuaren laburpen-txartela sortzen du (oinarri gisa emanda dator; egokitu zuen eremuetara).
- `toStorageObject()`: `JSON.stringify()` bidez gordetzeko prest dagoen objektu arrunta itzultzen du.
- `static fromStorageObject(obj)`: objektu arrunt horretatik `Item` bat berreraikitzen du.

Ondoren, bilaketako emaitza bakoitza `Item` klaseko instantzia bihurtu eta `item.toHTML()` erabiliz erakutsi. Ez idatzi datuak eskuz `HTML`an. Txartel bakoitza klik eginez hautatu ahal izango da.

Hautatutako elementua: emaitza batean klik egitean, erakutsi elementu horren propietate guztiak panelean, ez soilik txartelean agertzen direnak. Gehitu balorazioa (1-5) eta iruzkina sartzeko eremuak, eta "`Gehitu bildumara`" botoia. Eduki hori sortzeko, `Item` klaseari `toXehetasunHTML()` metodoa gehi diezaiokezue, adibidez; halaber, zerrendan hautatutako txartela nabarmendu.

1.3. Bilduman gordetzea (`js/bildumaStorage.js`)

Garatu `bildumaStorage` klasea, metodo estatikoekin, bilduma `localStorage`-n gordetzeko:

Metodoak	Zer egiten du
`denakLortu()`	`localStorage`-n gordetako array-a irakurtzen du, edo `[]` itzultzen du ezer gordeta ez badago.
`gorde(item)`	`item.toStorageObject()` gehitzen dio `array`-ari eta `JSON.stringify()` erabiliz gordetzen du; `ID`aren arabera bikoiztuak saihesten ditu.
`itemBezalaLortu()`	`Item` instantziak itzultzen ditu `Item.fromStorageObject()` eta `map()` erabiliz.
`ezabatu(id)`	`ID` hori duen elementua kentzen du `filter()` erabiliz.
`hustu()`	Bilduma osoa ezabatzen du.

2. Taula: bildumaStorage Klasean garatu beharreko metodoak

Hautatutako elementuaren panelean, erabiltzaileak balorazioa (1-5) eta iruzkina sar ditzake. Paneleko "`Gehitu bildumara`" botoiak datu horiekin `Item` bat sortu eta `bildumaStorage.gorde(item)` deituko du. `ezabatu(id)` eta `hustu()` metodoak 3. urratsean erabiliko dituzue.

`Item` klaseko instantzietan eta objektuetan bilaketako balioak, balorazioa, gordetze-data eta iruzkina gordeko dira. Adibidez, 5eko balorazioa duen Pikachu bat:

```
{
  "id": 25,
  "izena": "pikachu",
  "irudia": "https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/25.png",
  "motaks": ["electric"],
  "altuera": 0.4,
  "pisua": 6,
  "balorazioa": 5,
  "iruzkina": "Nire gogokoena",
  "gordetakoData": "2026-10-16T10:42:00.000Z"}
```

2. urratsa: Nire bilduma (`bilduma.html`)

Atal honek gordetako bilduma erakusten du, eta programazio funtzionala lantzeko erabiliko duzue. Talde bakoitzak API desberdina erabiltzen duenez, zuek erabakiko duzue zein eremuri aplikatu funtzio bakoitza, eta memorian justifikatuko duzue.

- **Iragazkia** `array.filter()` erabiliz, erabiltzailearen eremu baten gainean (balorazioa, iruzkina) edo APIaren eremu baten gainean (kategoria, mota, urtea...).
- **Ordenatzea** `array.sort()` erabiliz, gutxienez bi eremuren arabera (adibidez, balorazioa, izenburua, urtea edo gordetze-data), goranzkoan eta beheranzkoan. Iragazitako emaitzari aplikatuko zaio.
- **Errenderizatzea** `array.map()` erabiliz: iragazitako eta ordenatutako `Item` bakoitza `HTML` bihurtzen da `toHTML()` bidez.

Xehetasunen panela `bilduma` atalean ere: gordetako elementu batean klik eginez gero, haren propietate guztiak ikusteko aukera eman.

3. urratsa: Bilduma kudeatzea (`kudeatuBilduma.html`)

Atal honek Nire bilduma atalaren bilduma bera erakusten du, baina hura aldatzeko aukerekin. Elementu bakoitzaren ekintzetan gertaeren delegazioa erabiliko da:

- **Elementu bat ezabatzea**: "Ezabatu" botoiak `bildumaStorage.ezabatu(id)` deitzen du eta ikuspegia eguneratzen du.
- **Elementu bat aldatzea**: haren balorazioa (1-5) eta iruzkina editatu eta aldaketak `localStorage`-n gordetzea. Horretarako, gehitu `eguneratu(item)` metodoa `bildumaStorage`-ri.
- **Bilduma hustea**: botoia sakatzean, aurretik `confirm()` bidez berrespena eskatu.

4. urratsa: Estatistikak (`estatistikak.html` gehitzea)

Atal hau ez dago emanda: zuek sortu behar duzue. Jarraitu lehendik dauden atalen eredua: gehitu botoia `<nav>` barruan, `<section>` bat `index.html`-n, `templates/estatistikak.html` txantiloia eta sarrera bat `app.js`-ko `ATALAK` `array`-an.

Horretarako, ulertu `app.js` nola dabilen: txantiloi bakoitza `fetch` bidez kargatu eta dagokion edukiontzian txertatzen du; menuak atalak erakutsi eta ezkutatzen ditu orria berriro kargatu gabe, eta aktibo dagoena `active` klasearekin markatzen du.

Atalak `array.reduce()` erabiliz egindako kalkuluak erakutsiko ditu:

- Gordetako elementuen kopuru osoa.
- Batez besteko balorazioa.
- Balorazioen banaketa edo kategoria bakoitzeko kopurua.
- Balorazio onena eta txarrena duen elementua.
- Zuen domeinuan bururatzen zaizkizuen beste aukera batzuk.

Adibidez, Pokemonen kasuan:

- Mota bakoitzeko zenbat Pokemon dauden.
- Zenbat Pokemon diren legendarioak.

...

5. urratsa: Ezarpenak (`ezarpenak.html` gehitzea)

Errepikatu aurreko urratseko eredua Ezarpenak atala sortzeko (`templates/ezarpenak.html`):

- Modu iluna/argia (derrigorrezkoa): erabiltzailearen hobespena `localStorage`-n gordetzen duen etengailua.
- Gainerako ezarpen-aukerak gehigarriak dira.

6. urratsa: Gehigarriak, IA erabili gabe

Gehigarriek derrigorrezkoaren gainetik nota igotzeko aukera ematen dute. Pentsatu eta garatu hobekuntzak 5. urratseko aplikazioan oinarrituta. Memorian oso argi azaldu behar da zer funtzionalitate edo alderdi gehitu dituzuen hobekuntza gisa. Gehigarriek 1,5 puntu balio dute; ondo landuz gero, balio hori gaindi dezakete eta beste urrats batzuetan galdutako puntuak konpentsatu.

Ideia batzuk:

- Bilaketen historia eta osatze automatikoa: bilaketa bakoitza datarekin gordetzen du eta berriro bilatzean iradokitzen du; horretarako, ‹datalist› natiboa edo zuen zerrenda propioa erabil dezakezue.
- Bilduma `.json` fitxategi gisa esportatzea eta inportatzea.
- Estatistiketan banaketa-barrak erakustea, `JavaScript`ek kalkulatutako zabalera duen `<div>` elementua erabiliz.
- Konparatzailea: bildumako bi elementu edo gehiago hautatu eta elkarren ondoan erakustea.
- “Harritu nazazu” botoia: APIko ausazko elementu bat gehitzen dio bildumari; ausazko `ID`ak erabil ditzakezue.

7. urratsa: Gehigarriak, IA erabiliz

- Web-orriaren estiloak eta birdiseinua.
- IA tresna baten laguntzaz garatutako funtzionalitate anbiziotsuagoak.

Gehigarriek 1,5 puntu balio dute eta, aurreko urratsean bezala, ondo landuz gero puntu gehiago lor ditzakete.

IA erabiltzea urrats honetan bakarrik dago baimenduta eta ebaluatuko da, eta memorian dokumentatu behar da. IA bidez sortutako kodea ulertu gabe kopiatzea zigortu egingo da gainerako urratsetan; urrats honetan, ez.

Zati hau “`milestone2_frontEnd`” entregan aurkeztuko da. Ez duzue kodea hutsetik garatu behar; hemen egin beharrekoa egiaztatzea da. Bertsio honek ez badu funtzionatzen edo erroreak baditu, 0 puntu jasoko ditu.

8. urratsa: Memoria

Memoriak puntu 1 balio du, eta talde bakoitzak memoria bakarra entregatuko du. eGelan txantiloi bat duzue eskuragarri.

Ebaluazioa eta entregak

Front-End zatia honela ebaluatuko da:

Urratsa	Edukia	Puntuak	Entrega
0	APIa hautatzea (fitxa)	1	1. entrega
1 eta 2	Bilatzailea eta Nire bilduma	3	`Milestone 1`
3	Bilduma kudeatzea	0,75	`Milestone 1`
4	Estatistikak	1	`Milestone 1`
5	Ezarpenak	0,75	`Milestone 1`
6	Gehigarriak, IA erabili gabe	1,5	`Milestone 1`
7	Gehigarriak, IA erabiliz	1+	`Milestone 2`
8	Memoria	1+	`Milestone 1 eta 2`

3. Taula: Ebaluazio taula, puntuazioa urratsez-urrats

GUZTIRA: 10 puntu.

Ondo landutako gehigarriek beren balioa baino puntu gehiago eman ditzakete, eta beste urrats batzuetan galdutako puntuak konpentsatu. Zati honetako gehieneko nota 10 da.

Milestone 1 (IArik gabe)

1etik 5era bitarteko urratsak eta IA erabili gabeko gehigarriak entregatuko dituzue, fase honetako memoriarekin batera. Lan egin main adarrean eta, entregatzean, finkatu egoera `Git`-eko tag batekin. Tag hori Back-End zatiaren abiapuntua izango da:

git tag -a milestone-1 -m "Front-End sin IA"

git push origin milestone-1

Milestone 2 (IA erabiliz)

IA bidezko gehigarriak beste adar batean egingo dira, Milestone 1eko tag-etik abiatuta, IA erabiliz egindakoa argi bereizteko. Entregatzean, sortu tag-a:

```
git switch -c frontend-ia milestone-1

git push -u origin frontend-ia

git tag -a milestone-2 -m "Front-End eta IA"

git push origin milestone-2
```